Practica Experimental: Emparejamiento Estable

Jalton Efrain Jara Neira *(Centro de Investigación en Computación, IPN, jjarab25@cic.ipn.mx)*

***Resumen*—El presente reporte hace un análisis experimental del algoritmo de Gale-Shapley para resolver el Problema del Emparejamiento Estable (SMP). El estudio se fundamenta en la teoría de estabilidad propuesta en 1962, orientada a procesos de asignación auto-ejecutables que evitan incentivos individuales de ruptura. Se implementó una versión del algoritmo en lenguaje Python 3.14 utilizando técnicas para reducir la búsqueda de preferencias a tiempo constante $O(1)$, evitando el cambio de la complejidad asintótica. La metodología consistió en la generación automatizada de archivos de entrada con dos tasas de crecimiento $N = 2^i$ y $N = 600i$. Las pruebas se realizaron con un procesador AMD Ryzen 7 y 32 GB de RAM, logrando procesar exitosamente hasta $N = 2^{14}$ parejas. Los resultados se visualizaron en escalas log-log y lineal, confirmando una complejidad temporal cuadrática $O(n^2)$.**

I. Introducción

El problema del emparejamiento estable tuvo origen en 1962 cuando dos economistas matemáticos, David Gale y Lloyd Shapley, plantearon si se podía diseñar un proceso de admisión universitaria o un proceso de contratación laboral que fuera auto-ejecutable.

Plantearon, si un grupo de amigos de tercer año estudiantes de ciencias de la computación, empiezan a postularse a pasantías de verano. En este proceso de solicitud la interacción se da entre dos partes diferenciadas, los estudiantes (o solicitantes) y las empresas (o empleadores). Primeramente, cada solicitante forma una lista de preferencia sobre la empresa a la que decide postular y posteriormente cada empresa forma un orden de preferencia con forme empiezan a llegar las solicitudes, de forma tal que las empresas extienden ofertas a los candidatos quienes eligen que ofertas aceptar.

Gale y Shapley consideran que este proceso puede salirse de control debido a que si un solicitante decide su postulación en base a su propio interés iría cambiando de empresa en base se le vayan presentando nuevas ofertas, del mismo modo alguna empresa inescrupulosa podría ir retirándole solicitantes en base a su propio beneficio, creando un ambiente de inestabilidad originado por personas actuando de acuerdo a su propio interés poniendo el riesgo el sistema. Ellos consideraron que para solucionar este problema lo mejor sería establecer un mecanismo de control que permitiera impedir que las ofertas sean retiradas y redirigidas si se establece que tanto empleadores como solicitantes establecen una de preferencias de modo tal que: para cada empleador E y cada solicitante A que no esté programado para trabajar para E, se cumpla al menos una de las siguientes dos condiciones:

- E prefiere a cada uno de sus candidatos aceptados por encima de A; o
- A prefiere su situación actual antes que trabajar para el empleador E.

Y si esto se cumple, el interés propio individual evitará que se realice cualquier acuerdo entre solicitante y empleador fuera de lo legal, llegando a tener un resultado estable. [1]

II. Objetivo

El objetivo de esta práctica es comprobar experimentalmente el comportamiento asintótico del algoritmo de Gale - Shapley que encuentra el emparejamiento estable de dos conjuntos de hombres y mujeres cuya cardinalidad de ambos es N. Es decir, resolver el SMP (Stable Matching Problem) en su versión básica.

III. Descripción de actividades

A. Metodología

Se diseñó un script en Python para generar automáticamente los M = 25 archivos de entrada. Finalmente, de acuerdo a las características del hardware sólo se generaron M’ = 17 archivos.

Se utilizó una tasa de crecimiento de los datos de $N = 2^i$ para observar el comportamiento temporal.

La estructura de cada archivo de entrada esta conformada por N hombres y N mujeres, donde el tamaño de N y el rol proponente queda establecido en la primera línea, seguido de las listas de preferencias de cada miembro desordenados aleatoriamente.

Para el diseño del algoritmo principal como consideración importante se implementó una lista de preferencias de tal manera indicara la posición de la preferencia, evitando el uso del método

.index(p) que podía incrementar la complejidad del algoritmo a $O(n^3)$. La propuesta y extracción de datos se hizo a través del método pop() en la lista de preferencias, asegurando que no se repitan las propuestas, reduciendo el espacio de búsqueda en cada paso y manteniendo el control del número máximo de operaciones.

Se diseño un script en Python que permitió la medición de los tiempos que toma leer los datos de entrada de cada archivo generado, el tiempo que demora en imprimir los resultados de emparejamiento y finalmente el tiempo que demora en encontrar el emparejamiento, es decir el tiempo de ejecución del algoritmo de Gale-Shapley.

La medición se hizo a través de la biblioteca ***time*** de Python. Cada uno de estos tiempos se imprimieron en una tabla en formato .csv, al mismo tiempo se hizo una gráfica en escala logarítmica del tiempo de ejecución en segundos vs el número N de parejas de cada archivo de entrada.

Finalmente, se realizó una segunda generación de archivos L = 25, con una tasa de crecimiento de N = 600i. Y se procedió a graficarlos esta vez en escala lineal del tiempo de ejecución (s) vs el número de parejas (n).

B. Herramientas

Software:
- IDE de Visual Studio Code
- Lenguaje Python V 3.14
- Bibliotecas: ***Matplotlib, Time, Sys y Random.***

Hardware:
- Procesador AMD Ryzen 7 Pro 7840U y 32 GB de memoria RAM DDR4.

IV. Resultados

TABLA I
TIEMPOS DE EJECUCIÓN

Archivo	N	Tiempo de Lectura (s)	Tiempo de Impresión (s)	Tiempo de Algoritmo (s)
1	2	0.0001783	8.00E-07	9.80E-06
2	4	0.0001127	8.00E-07	7.90E-06
3	8	0.0001127	1.00E-06	1.69E-05
4	16	0.000171	1.50E-06	3.65E-05
5	32	0.0003081	2.40E-06	0.0001149
6	64	0.0006686	5.50E-06	0.0004285
7	128	0.0015421	8.00E-06	0.0012307
8	256	0.0047889	1.56E-05	0.0049618
9	512	0.03153	4.30E-05	0.020274
10	1024	0.0791506	6.69E-05	0.0933218
11	2048	0.3949011	0.000191	0.3594762
12	4096	1.4953332	0.0007069	1.4035489
13	8192	5.9810518	0.0020734	5.9692072
14	16384	33.3745777	0.005572	149.6950652

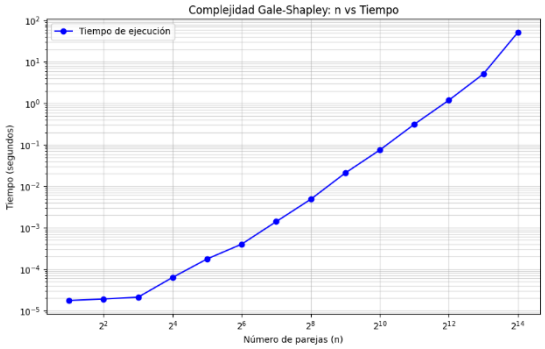

Fig. 1.Complejidad Temporal del algoritmo Gale – Shapley, escala log-log.

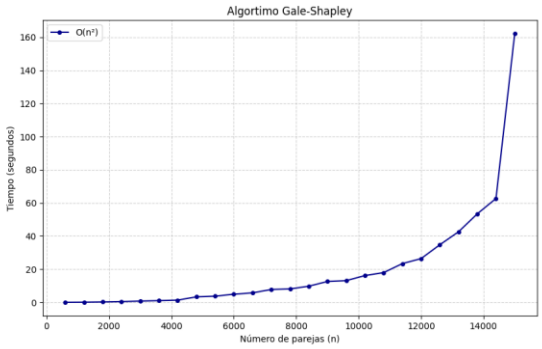

Fig. 2.Complejidad Temporal del algoritmo Gale – Shapley, escala lineal.

V. Análisis de los Resultados

El análisis de los datos obtenidos revela una correlación directa entre el incremento exponencial de N y el tiempo de respuesta del sistema. A continuación, se detalla el análisis de los resultados obtenidos en la experimentación realizada:

A. Comportamiento de los tiempos de ejecución

En la Tabla I se identifica que para valores de N menores a 1024, el tiempo de ejecución es prácticamente despreciable (menor a 0.05 segundos), lo que indica que, para escalas pequeñas, los tiempos fijos del lenguaje Python debido a la inicialización del intérprete y la carga de bibliotecas predominan sobre el algoritmo. Sin embargo, a partir de N = 2^{12}, el crecimiento comienza a manifestar su comportamiento cuadrático.

B. Análisis de la gráfica logarítmica

Elegimos utilizar una grafica en escala logarítmica debido a que la cantidad de datos de entrada es lo suficientemente grande para tener dificultades de tamaño de escala para visualizar de forma completa la curva, y la escala logarítmica nos permite que cada paso sea una multiplicación del anterior, facilitando el proceso de dibujo.

Al graficar los datos en una escala logarítmica, la pendiente de la línea resultante es aproximadamente $m \approx 2$. En términos de análisis asintótico, esto confirma que la función de tiempo

sigue la forma $T(n) = c.n^2$, donde c es una constante que depende de la arquitectura del computador utilizado. La desviación ligera en la rectitud de la línea para valores muy altos de N se debe al incremento en el tiempo de acceso a la memoria pudiendo haber cache misses al momento de la lectura de los archivos y la gestión de memoria virtual del sistema operativo.

C. Limitaciones de Hardware y Memoria

Un hallazgo importante de la práctica fue la imposibilidad de generar y procesar las 25 entradas sugeridas originalmente para entradas de la forma $N = 2^i$. Se pudieron generar M' = 17 archivos, es decir hasta una entrada de $N = 2^{17}$ parejas y se pudieron procesar M'' = 14 archivos, es decir hasta una entrada de $N = 2^{14}$ parejas. Se experimentaron los siguientes cuellos de botella:

Desbordamiento de memoria RAM, Python almacena los diccionarios de preferencias en memoria dinámica. Al ser una matriz de N x N, el consumo de memoria crece cuadráticamente, superando la capacidad física disponible del equipo.

Gestión de archivos I/O, el tamaño de los archivos .txt generados superó el umbral de manejo del sistema de archivos, provocando que el tiempo de lectura fuera significativamente mayor que el tiempo de ejecución del algoritmo propiamente dicho.

D. Análisis de la segunda gráfica

En esta segunda grafica se puede apreciar el crecimiento cuadrático del tiempo de ejecución, al inicio de n = 600 a n = 3000 la curva tiene una pendiente muy leve para valores bajos de n, mientras que de n = 12000 a n = 15000 la pendiente se vuelve mucho más vertical. La curva que se observa es una parábola de apertura positiva de la forma $T(n) = c.n^2$, al no ser una curva logarítmica se aprecia de mejor manera el crecimiento explosivo del comportamiento cuadrático.

VI. CONSLUSIONES

Se comprobó experimentalmente que el algoritmo de Gale - Shapley cumple con una complejidad temporal $O(n^2)$. La estabilidad de los emparejamientos se logró en todos los casos procesados, demostrando la robustez del diseño.

La implementación de una estructura de datos para el ranking de las mujeres permitió reducir la búsqueda de preferencias de un costo lineal $O(n)$ a un costo constante $O(1)$, factor determinante para mantener la complejidad cuadrática del algoritmo.

La experimentación demostró que la complejidad espacial $O(n^2)$ en el almacenamiento de preferencias es un factor crítico tanto como la complejidad temporal. Siendo imperativo utilizar representaciones de datos más compactas para superar las limitaciones de la memoria RAM.

Se concluye que el tiempo de lectura y pre-procesamiento de datos empieza a ser considerable en la evaluación de valores grandes de n, llegando incluso a superar el tiempo de cómputo del emparejamiento estable.

VII. BIBLIOGRAFÍA

[1] J. Klenberg, and E. Tardos, Algorithm Design 1st *ed*. Cornell University, 2006, pp. 21-30.